

```

import { useState, useCallback } from 'react';
import Papa from 'papaparse';
import * as XLSX from 'xlsx';
import { useDropzone } from 'react-dropzone';
import { UploadCloud, CheckCircle, AlertCircle, FileSpreadsheet, Loader2 } from 'lucide-react';
import { clsx, type ClassValue } from 'clsx';
import { twMerge } from 'tailwind-merge';
function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
type Step = 'UPLOAD' | 'PREVIEW' | 'PROCESSING' | 'RESULT';
export default function Home() {
  const [step, setStep] = useState<Step>('UPLOAD');
  const [file, setFile] = useState<File | null>(null);
  const [allRecords, setAllRecords] = useState<any[]>([]);
  const [previewData, setPreviewData] = useState<{ headers: string[], rows: any[][] }>({ headers: [], rows: [] });
  const [resultData, setResultData] = useState<{ totalImported: number; totalSkipped: number; records: any[] } | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [progress, setProgress] = useState<{ current: number; total: number }>({ current: 0, total: 0 });
  const parseFilePreview = useCallback((file: File) => {
    if (file.name.endsWith('.csv')) {
      Papa.parse(file, {
        header: true,
        skipEmptyLines: true,
        complete: (results) => {
          if (results.data && results.data.length > 0) {
            const records = results.data as any[];
            setAllRecords(records);
            // Extract headers and rows for preview
            const headers = Object.keys(records[0] || {});
            const rows = records.slice(0, 50).map(record => headers.map(h => record[h]));
            setPreviewData({ headers, rows });
            setStep('PREVIEW');
            setError(null);
          } else {
            setError('The uploaded CSV appears to be empty.');
```

```

setPreviewData({ headers, rows });
setStep('PREVIEW');
setError(null);
} else {
setError('The uploaded Excel file appears to be empty.');
```

```

}
```

```

} catch (err: any) {
setError(err.message || 'Failed to parse Excel file.');
```

```

}
```

```

};
reader.onerror = () => setError('Failed to read the file.');
```

```

reader.readAsBinaryString(file);
```

```

}
```

```

}, []);
```

```
const onDrop = useCallback((acceptedFiles: File[]) => {
```

```
  if (acceptedFiles.length > 0) {
```

```
    const selectedFile = acceptedFiles[0];
```

```
    const isCsv = selectedFile.type === 'text/csv' || selectedFile.name.endsWith('.csv');
```

```
    const isExcel = selectedFile.name.match(/\.xlsx?$/i) || selectedFile.type.includes('spreadsheetml') ||
```

```
    selectedFile.type.includes('ms-excel');
```

```
    if (isCsv || isExcel) {
```

```
      setFile(selectedFile);
```

```
      parseFilePreview(selectedFile);
```

```
    } else {
```

```
      setError('Please upload a valid CSV or Excel file.');
```

```
    }
```

```
  }
```

```
}, [parseFilePreview]);
```

```
const { getRootProps, getInputProps, isDragActive } = useDropzone({
```

```
  onDrop,
```

```
  accept: {
```

```
    'text/csv': ['.csv'],
```

```
    'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet': ['.xlsx'],
```

```
    'application/vnd.ms-excel': ['.xls']
```

```
  },
```

```
  maxFiles: 1
```

```
});
```

```
const handleConfirm = async () => {
```

```
  if (!file || allRecords.length === 0) return;
```

```
  setStep('PROCESSING');
```

```
  setError(null);
```

```
  setProgress({ current: 0, total: allRecords.length });
```

```
  const BATCH_SIZE = 20;
```

```
  const allImported: any[] = [];
```

```
  let hasError = false;
```

```
  for (let i = 0; i < allRecords.length; i += BATCH_SIZE) {
```

```
    if (hasError) break;
```

```
    const batch = allRecords.slice(i, i + BATCH_SIZE);
```

```
    try {
```

```
      const response = await fetch('/api/upload', {
```

```
        method: 'POST',
```

```
        headers: {
```

```
          'Content-Type': 'application/json',
```

```
        },
```

```
        body: JSON.stringify({ batch }),
```

```
      });
```

```
      let data;
```

```

try {
const textResponse = await response.text();
try {
data = JSON.parse(textResponse);
} catch (e) {
console.error('Non-JSON response:', textResponse.substring(0, 200));
throw new Error(`Server returned an invalid response (Status ${response.status}). This could be a
timeout or server error.`);
}
} catch (parseError: any) {
throw new Error(parseError.message || `Server returned an invalid response (Status
${response.status}).`);
}
if (!response.ok) {
throw new Error(data?.error || `Failed to process batch (Status ${response.status})`);
}
allImported.push(...(data.records || []));
setProgress({ current: Math.min(i + BATCH_SIZE, allRecords.length), total: allRecords.length });
} catch (err: any) {
setError(err.message || 'An error occurred during import.');
```

setStep('PREVIEW'); // Let them try again

hasError = true;

break;

}

}

if (!hasError) {

setResultData({

totalImported: allImported.length,

totalSkipped: allRecords.length - allImported.length,

records: allImported,

});

setStep('RESULT');

}

};

const reset = () => {

setStep('UPLOAD');

setFile(null);

setPreviewData({ headers: [], rows: [] });

setResultData(null);

setError(null);

setAllRecords([]);

setProgress({ current: 0, total: 0 });

};

return (

<div className="min-h-screen bg-[#F8FAFC] font-sans text-slate-900 selection:bg-indigo-100

selection:text-indigo-900">

{/* Header */}

<header className="bg-white border-b border-slate-200 sticky top-0 z-50">

<div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 h-16 flex items-center justify-between">

<div className="flex items-center gap-3">

<div className="w-8 h-8 bg-indigo-600 rounded-lg flex items-center justify-center shadow-sm">

<FileSpreadsheet className="w-5 h-5 text-white" />

</div>

<h1 className="text-xl font-bold tracking-tight text-slate-900">GrowEasy Data Sync</h1>

</div>

{step !== 'UPLOAD' && (

<button

```

onClick={reset}
className="text-sm font-semibold text-slate-500 hover:text-slate-900 transition-colors"
>
Start New Import
</button>
)}
</div>
</header>
<main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8 h-[calc(100vh-4rem)]">
<div className="h-full flex flex-col max-w-5xl mx-auto">
{error && (
<div className="mb-6 p-4 bg-rose-50 border border-rose-200 rounded-xl flex items-start gap-3 shrink-0">
<AlertCircle className="w-5 h-5 text-rose-600 shrink-0 mt-0.5" />
<div>
<h3 className="text-sm font-bold text-rose-800">Import Error</h3>
<p className="text-sm text-rose-600 mt-1">{error}</p>
</div>
</div>
)}
{step === 'UPLOAD' && (
<div className="flex-1 flex items-center justify-center">
<div
{...getRootProps()}
className={cn(
"w-full max-w-2xl p-12 border-2 border-dashed rounded-3xl text-center cursor-pointer transition-all
duration-200 ease-in-out flex flex-col items-center justify-center bg-white",
isDragActive ? "border-indigo-500 bg-indigo-50/50 scale-[1.02]" : "border-slate-200
hover:border-indigo-300 hover:bg-slate-50"
)}
>
<input {...getInputProps()} />
<div className="w-16 h-16 bg-indigo-50 rounded-full flex items-center justify-center mb-6">
<UploadCloud className="w-8 h-8 text-indigo-600" />
</div>
<div>
<p className="text-xl font-bold text-slate-700">Drag & drop your file here</p>
<p className="text-sm text-slate-500 mt-1">or click to browse from your computer</p>
</div>
<div className="mt-6 flex items-center gap-2 text-xs font-medium text-slate-400">
<span>Supported formats: .csv, .xlsx, .xls</span>
</div>
</div>
)}
{step === 'PREVIEW' && (
<div className="flex-1 bg-white rounded-2xl shadow-sm border border-slate-200 flex flex-col overflow-hidden">
<div className="p-4 border-b border-slate-100 flex items-center justify-between bg-slate-50/50 shrink-0">
<div>
<h2 className="text-sm font-bold text-slate-700 uppercase tracking-wide">Data Preview</h2>
<p className="text-[11px] text-slate-500 mt-1">Reviewing the first 50 rows of <span
className="font-medium text-slate-700">{file?.name}</span></p>
</div>
<button
onClick={handleConfirm}

```

```

className="px-4 py-2 text-sm font-semibold text-white bg-indigo-600 hover:bg-indigo-700
rounded-lg shadow-sm transition-all flex items-center gap-2"
>
<CheckCircle className="w-4 h-4" />
Confirm & Import
</button>
</div>
<div className="overflow-auto flex-1 relative">
<table className="w-full text-left border-collapse">
<thead className="sticky top-0 bg-white shadow-[0_1px_0_rgba(0,0,0,0.05)] z-10">
<tr className="text-[11px] font-bold text-slate-400 uppercase">
<th className="px-6 py-4 border-b border-slate-100">#</th>
{previewData.headers.map((header, i) => (
<th key={i} className="px-6 py-4 border-b border-slate-100 whitespace-nowrap">
{header || `Column ${i + 1}`}
</th>
)))}
</tr>
</thead>
<tbody className="text-[13px] divide-y divide-slate-100">
{previewData.rows.map((row, rowIndex) => (
<tr key={rowIndex} className="hover:bg-slate-50/50">
<td className="px-6 py-4 text-slate-400">{rowIndex + 1}</td>
{previewData.headers.map( (_, colIndex) => (
<td key={colIndex} className="px-6 py-4 text-slate-700 truncate max-w-[200px]"
title={row[colIndex]}>
{row[colIndex] || <span className="text-slate-300">-</span>}
</td>
)))}
</tr>
)))}
</tbody>
</table>
</div>
</div>
)}
{step === 'PROCESSING' && (
<div className="flex-1 bg-white rounded-2xl shadow-sm border border-slate-200 flex flex-col
items-center justify-center p-12 text-center">
<Loader2 className="w-16 h-16 text-indigo-600 animate-spin mb-6" />
<h2 className="text-2xl font-bold text-slate-900 mb-2">Extracting CRM Data</h2>
<p className="text-slate-500 max-w-md mx-auto text-sm mb-8">
Our AI is currently analyzing your records, mapping varying column names, and structuring the
leads for GrowEasy CRM.
</p>
<div className="w-full max-w-md">
<div className="flex justify-between text-xs font-semibold text-slate-500 mb-2">
<span>Processing batch...</span>
<span>{progress.current} / {progress.total}</span>
</div>
<div className="w-full h-2 bg-slate-100 rounded-full overflow-hidden">
<div
className="h-full bg-indigo-600 transition-all duration-300 ease-out"
style={{ width: `${progress.total > 0 ? (progress.current / progress.total) * 100 : 0}%` }}
></div>
</div>
</div>

```

```

</div>
  })
  {step === 'RESULT' && resultData && (
    <>
      <section className="grid grid-cols-2 md:grid-cols-4 gap-4 shrink-0">
        <div className="bg-white p-5 rounded-2xl shadow-sm border border-slate-200">
          <p className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-1">Processed</p>
          <p className="text-3xl font-light text-indigo-600">{resultData.totalImported +
            resultData.totalSkipped}<span className="text-sm font-normal text-slate-400">leads</span></p>
        </div>
        <div className="bg-white p-5 rounded-2xl shadow-sm border border-slate-200">
          <p className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-1">Successfully
            Mapped</p>
          <p className="text-3xl font-light text-emerald-600">{resultData.totalImported}</p>
        </div>
        <div className="bg-white p-5 rounded-2xl shadow-sm border border-slate-200">
          <p className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-1">Skipped (No
            Contact)</p>
          <p className="text-3xl font-light text-rose-500">{resultData.totalSkipped}</p>
        </div>
        <div className="bg-white p-5 rounded-2xl shadow-sm border border-slate-200">
          <p className="text-xs font-bold text-slate-400 uppercase tracking-wider mb-1">AI Confidence</p>
          <p className="text-3xl font-light text-slate-700">~98%</p>
        </div>
      </section>
      <div className="flex-1 bg-white rounded-2xl shadow-sm border border-slate-200 flex flex-col
        overflow-hidden mt-4">
        <div className="p-4 border-b border-slate-100 flex items-center justify-between bg-slate-50/50
          shrink-0">
          <h2 className="text-sm font-bold text-slate-700 uppercase tracking-wide">AI Extraction
            Results</h2>
          <div className="flex gap-2">
            <span className="px-2 py-1 bg-emerald-50 text-emerald-700 text-[10px] font-bold rounded border
              border-emerald-100">CLEAN DATA</span>
          </div>
        </div>
        <div className="overflow-auto flex-1 relative">
          <table className="w-full text-left border-collapse">
            <thead className="sticky top-0 bg-white shadow-[0_1px_0_rgba(0,0,0,0.05)] z-10">
              <tr className="text-[11px] font-bold text-slate-400 uppercase">
                {resultData.records.length > 0 && Object.keys(resultData.records[0]).map((header) => (
                  <th key={header} className="px-6 py-4 border-b border-slate-100 whitespace-nowrap">
                    {header.replace(/_/g, ' ')}
                  </th>
                ))}
              </tr>
            </thead>
            <tbody className="text-[13px] divide-y divide-slate-100">
              {resultData.records.map((record, rowIndex) => (
                <tr key={rowIndex} className="hover:bg-slate-50/50">
                  {Object.entries(record).map(([key, value], colIndex) => {
                    let cellContent: any = value || <span className="text-slate-300">-</span>;
                    if (key === 'crm_status' && value) {
                      let style = "bg-slate-100 text-slate-600";
                      if (value === "GOOD_LEAD_FOLLOW_UP") style = "bg-amber-50 text-amber-700";
                      if (value === "BAD_LEAD") style = "bg-rose-50 text-rose-600";
                      if (value === "SALE_DONE") style = "bg-emerald-100 text-emerald-700";

```

```

cellContent = <span className={`px-2 py-1 rounded-full text-[10px] font-bold ${style}`}>{value as
string}</span>;
}
return (
<td key={collIndex} className="px-6 py-4 text-slate-700 truncate max-w-[200px]" title={value as
string}>
{cellContent}
</td>
);
}}
</tr>
)))
</tbody>
</table>
</div>
<div className="h-12 border-t border-slate-100 flex items-center justify-center bg-white shrink-0">
<button className="text-[11px] font-bold text-indigo-600 hover:underline uppercase
tracking-widest">View All {resultData.totalImported} Records</button>
</div>
</div>
</>
)}
</div>
</main>
</div>
);
}
import { NextResponse } from 'next/server';
import { GoogleGenAI, Type } from '@google/genai';
const ai = new GoogleGenAI({
apiKey: process.env.GEMINI_API_KEY,
// The SDK automatically handles retries, but we added a custom delay inside our route
httpOptions: {
timeout: 300000, // 5 minute timeout for batch AI processing
}
});
export async function POST(req: Request) {
try {
const { batch } = await req.json();
if (!batch || !Array.isArray(batch)) {
return NextResponse.json({ error: 'No batch provided' }, { status: 400 });
}
const prompt = `You are an AI CRM data extractor.
Map the provided JSON array of raw CSV records into the following CRM format fields:
created_at, name, email, country_code, mobile_without_country_code, company, city, state,
country, lead_owner, crm_status, crm_note, data_source, possession_time, description.
Rules:
1. Intelligently map unknown column names (like "Phone Number" or "ph") to the correct schema
fields.
2. If a record has NO email and NO mobile number, skip it (do not include it in the output array).
3. For crm_status, strictly use ONLY these exact values based on context:
GOOD_LEAD_FOLLOW_UP, DID_NOT_CONNECT, BAD_LEAD, SALE_DONE. If unknown,
leave empty.
4. Convert dates/times to ISO strings if possible for created_at and possession_time.
5. If data is missing for a field, leave it as an empty string.
6. Do NOT invent data. Only extract from the provided JSON.
7. Return only a JSON array of the successfully mapped objects.`;

```

```

`;
let retries = 5;
let success = false;
let result = [];
while (retries > 0 && !success) {
  try {
    const response = await ai.models.generateContent({
      model: "gemini-3.5-flash",
      contents: [
        { text: prompt },
        { text: JSON.stringify(batch) }
      ],
      config: {
        responseMimeType: "application/json",
        responseSchema: {
          type: Type.ARRAY,
          items: {
            type: Type.OBJECT,
            properties: {
              created_at: { type: Type.STRING },
              name: { type: Type.STRING },
              email: { type: Type.STRING },
              country_code: { type: Type.STRING },
              mobile_without_country_code: { type: Type.STRING },
              company: { type: Type.STRING },
              city: { type: Type.STRING },
              state: { type: Type.STRING },
              country: { type: Type.STRING },
              lead_owner: { type: Type.STRING },
              crm_status: { type: Type.STRING },
              crm_note: { type: Type.STRING },
              data_source: { type: Type.STRING },
              possession_time: { type: Type.STRING },
              description: { type: Type.STRING },
            },
          },
        },
      },
    });
    if (response.text) {
      result = JSON.parse(response.text.trim());
    }
    success = true;
  } catch (e: any) {
    retries--;
    if (retries === 0) {
      console.error("Batch failed after 5 retries", e);
      throw new Error(`AI processing failed: ${e.message || 'Unknown error'}`);
    }
    const delay = Math.pow(2, 5 - retries) * 2000;
    console.warn(`Batch processing failed, retrying in ${delay}ms... (Attempt ${5 - retries} of 4)`);
    e.message;
    await new Promise(resolve => setTimeout(resolve, delay));
  }
}
return NextResponse.json({
  records: result,

```

```
});  
} catch (error: any) {  
  console.error("Upload API error:", error);  
  return NextResponse.json(  
    { error: error.message || 'Failed to process file' },  
    { status: 500 }  
  );  
}  
}
```